Exploring the OSI Model and TCP/IP

The need for networking models

- A **network** is defined as having two or more computing devices interconnected, using a set of communication protocols (rules) that allow them to share a resource between themselves.
- A **resource** can be anything, such as a file on a centralized server, a multiplayer game on an online server, and even a network-connected printer.
- The largest network in the world is the **Internet.**
- Various organizations globally have the responsibility of ensuring its sustainability, availability, security, and scalability.
- The following are important organizations that play key roles on the internet:

Internet Society (ISOC), Internet Engineering Task Force (IETF), Internet Architecture Board (IAB), Internet Assigned Numbers Authority (IANA), Internet Corporation for Assigned Names and Numbers (ICANN).

- In the 1970s, the International Organization for Standardization (ISO) developed the OSI networking model for computer networks.
- The OSI model was designed to be a common standard for using networking protocols (rules) to allow intercommunication between devices that are connected over a network.
- A **network protocol** is simply the rules and guidelines that are used by a device to allow communication or the exchange of messages from one device to another.
- There are many network protocols, each of which has a different purpose and characteristic.

Exploring the OSI model

- The OSI model was originally developed to be an open networking model for computer networks to allow different devices to use a set of mutual protocols (rules) to allow communication between each other over a network.
- It is described as a reference model because it's not technically implemented on any networked devices.
- The OSI model contains a total of **seven layers** that describe how communication occurs between one device and another over a network.
- Each layer of the OSI model has a unique role and responsibility to ensure a message from a sender contains all the necessary details to be successfully delivered to the intended destination.

The following diagram shows the seven layers of the OSI model:

Layer	Name	Protocol Data Unit (PDU)
7	Application	Data
6	Presentation	
5	Session	
4	Transport	Segment
3	Network	Packet
2	Data Link	Frame
1	Physical	Bits

Fig. - OSI model

- At each layer of the OSI model, when a message exits at a specific layer, the message is commonly referred to as a **Protocol Data Unit** (**PDU**). A PDU is described as a single unit of data/information that can be transmitted from one host to another over a network.
- As the PDU is created at the **Application layer** of the OSI model of the host, it is referred to as **data**, which is the raw message.
- As the PDU travels down the OSI model, each of the lower layers is responsible for attaching additional information within a header onto the PDU to ensure proper addressing details are inserted to deliver the message. This process is called as **encapsulation**.
- When a host on the network receives the message, the PDU travels upward on the OSI model, where each layer extracts the message, removing the header information until the raw message is delivered to the Application layer on the recipient device. This process is called as **de-encapsulation**.

The following diagram shows an overview of the process of sending and receiving a message between two devices using the OSI model:

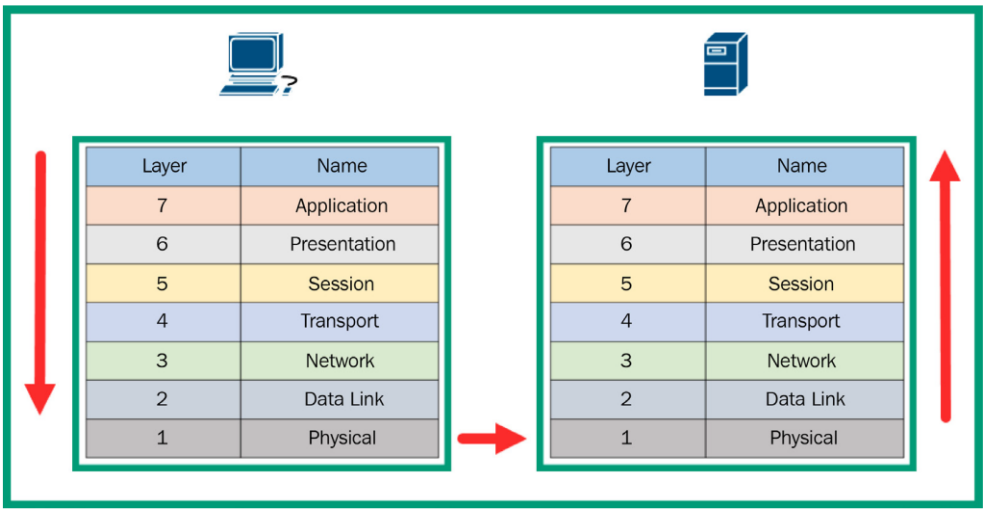


Fig. – Sending and receiving messages

- The **upper layers** of the OSI model, such as the **Application, Presentation, and Session layers** are designed to provide support for the **application's functionality**.

- The **lower layers** of the OSI model, such as the **Transport, Network, Data Link, and Physical layers** focus on inserting the addressing information needed **to deliver the datagram** to the destination.
- So, **lower layers** as having the responsibility of ensuring **end-to-end connectivity** between hosts over a network.

Application layer

- It is the closest to the end user. It provides an interface so that you can run the applications of a host such as a computer or even a smartphone to communicate with the underlying network protocols of the OSI model.
- The following screenshot shows a standard web browser using HTTPS as the Application layer protocol to communicate with the CompTIA web server:

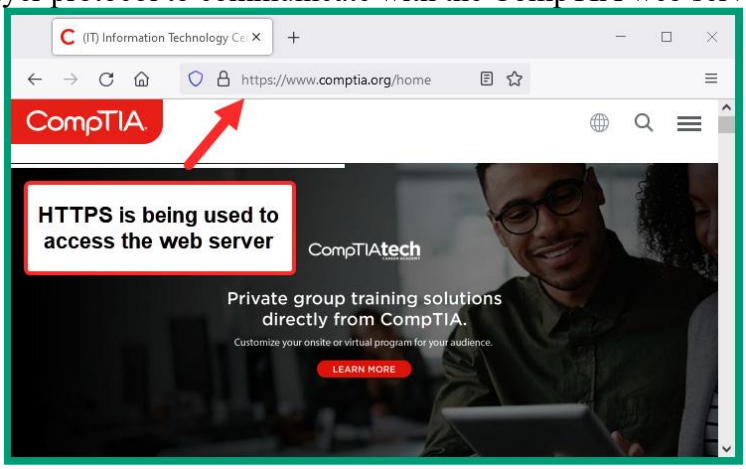


Fig. – Observing an Application layer protocol

- Installing a web browser on your computer allows your operating system to interact with the **Hypertext Transfer Protocol** (**HTTP**) and **Hypertext Transfer Protocol Secure** (**HTTPS**) protocols.
- Another example is using an email application such as Microsoft Outlook to interact/interface with the **Simple Mail Transfer Protocol** (**SMTP**), an application layer protocol that is responsible for sending email messages over a network.
- Each protocol uses its own set of rules and structure for creating a PDU.
- At the Application layer, the PDU contains only the raw data created by the application layer protocol and does not have any addressing information needed to be delivered to the intended recipient.
- At the Application layer, the PDU is known as **Data**.

Presentation layer

- It transforms system-dependent data (for example, ASCII or JPEG) into an independent format.
- This allows the Presentation layer on the receiving system to transform the data back into the system-dependent format (ASCII or JPEG) that the Application layer requires.

The following are the main responsibilities of the Presentation layer:

- Data formatting (encoding)
- Data compression
- Data encryption
- Data decryption

Session layer

The following are the core functions of the Session layer:

- Create/establish a session
- Maintain the session
- Terminate a session

Transport layer

- The Transport layer assigns a service port number to the PDU so that the receiving system will know how the Presentation layer should interpret and format the data.
- Then, the receiving system can read the data in the Application layer.

The following diagram shows a high-level visual representation of the client using HTTP to communicate with the same application layer protocol on the web server:

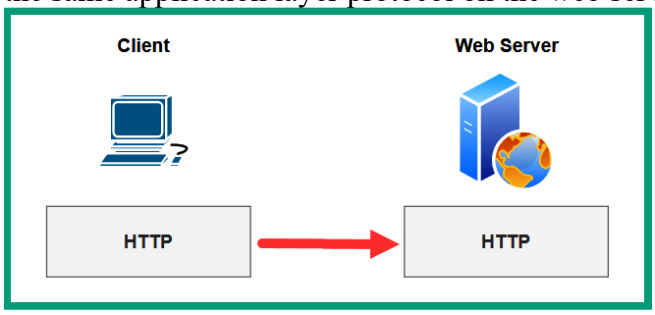


Fig. - Application layer protocol communication

- Within an operating system that supports TCP/IP, there are 65,535 service port numbers.
- The following diagram shows how these ports are categorized:

Port Range	Category
0 - 1,023	Well-known ports
1,024 - 49,151	Registered ports
49,152 - 65,535	Private/Dynamic ports

Well known ports- These are very common on a network. Some of these common application layer protocols are HTTP, HTTPS, and SMTP.

Registered ports-Their range belongs to users and organizations who have officially registered a service port number to operate on a custom build application or software.

Private/dynamic ports- Their range belongs to service ports that are temporarily used during communication, such as using a randomly generated service port on the sender's device as the source port.

- These service ports are logical ports within an operating system.
- Each service port number is logically mapped to an application layer protocol.

The following is a brief list of common application layer protocols and their corresponding service ports numbers:

Application Layer Protocol	Service Port Number
File Transfer Protocol (FTP)	20 & 21
Secure Shell (SSH)	22
Secure Copy (SCP)	22
SSH File Transfer Protocol (SFTP)	22
Telnet	23
Simple Mail Transfer Protocol (SMTP)	25
Domain Name System (DNS)	53
Hypertext Transfer Protocol (HTTP)	80
Hypertext Transfer Protocol Secure (HTTPS)	443

Fig. – Common application layer protocols

- Transport layer encapsulates (inserts) a layer 4 header onto the datagram that contains both the source and destination service port numbers.
- Once the layer 4 header is added to the datagram from the Application layer, the PDU is referred to as a **segment**.

The following diagram shows a segment at the Transport layer containing a source and destination service port number with the data received from the application layer protocol:

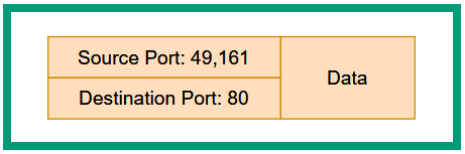


Fig. – Segment

- The source port number informs the recipient about the sender's return address.
- The destination service port number informs the destination device about which application layer protocol to deliver the message to.

The following diagram shows a visual representation of the client sending a message to the web server that is running HTTP as the application layer protocol on service port 80:

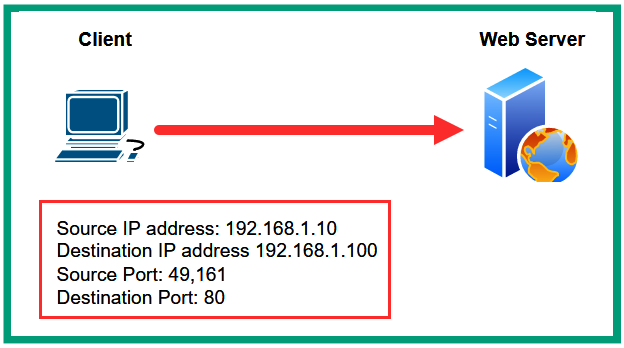


Fig. – HTTP Request message

The following diagram shows the addressing information used by the web server to respond to the client on the network:

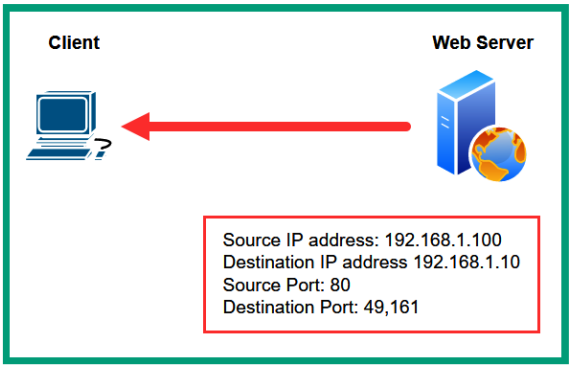


Fig. – HTTP Response message

- The Transport layer contains two protocols that assist with transporting and delivering datagrams over the network.
- These Transport layer protocols are as follows:
  - **Transmission Control Protocol (TCP)**
  - **User Datagram Protocol (UDP)**

Transmission Control Protocol

- It is a connection-oriented protocol that establishes a logical connection between the source and destination devices before exchanging messages over a network.
- This connection referred to as the **TCP three-way handshake**.

The following diagram shows a high-level overview of the TCP three-way handshake between two devices:

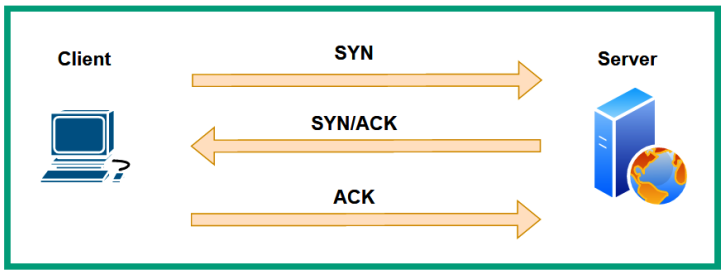


Fig. – TCP three-way handshake

The following is a breakdown of this process:

1. The client device wants to communicate with the server, so the client device sends a **synchronization (SYN)** message to the server. It is used to initiate a connection with the server. Within the SYN message, a randomly generated sequence number is created.

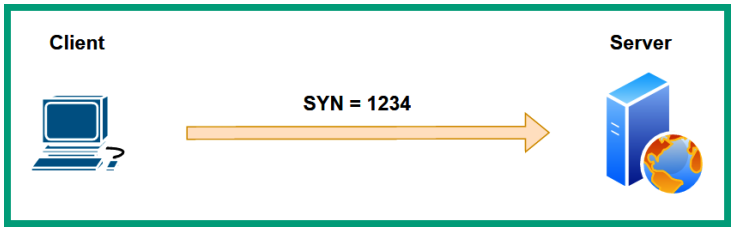


Fig. – SYN sequence number

2. The server receives the SYN message from the client and responds with an **acknowledgment (ACK)** message. Within the ACK message is an ACK sequence number; which is the client's sequence number + 1. The server also includes a SYN message within its response, containing a randomly generated sequence number to inform the client it also

wants to initiate a connection; this message is known as a **SYN/ACK**, as shown in the following diagram:

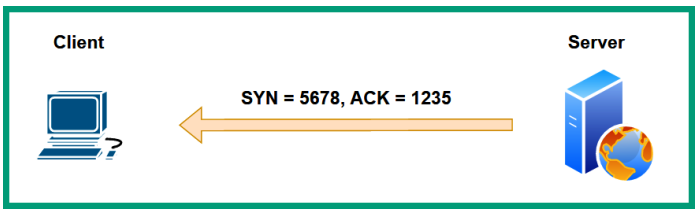


Fig. – SYN/ACK sequence number

3. The client receives the SYN/ACK message from the server and responds with an ACK message. The ACK message from the client contains an increment value of the SYN message received from the server, as shown in the following diagram:

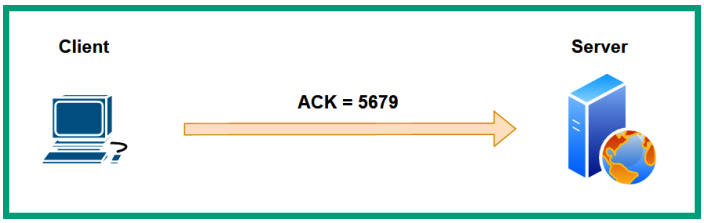


Fig. – ACK sequence number

TCP three-way handshake with sequence numbers

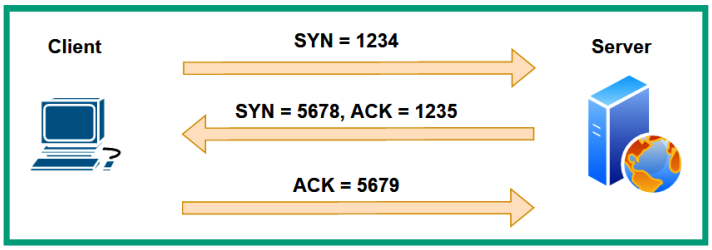


Fig. – TCP three-way handshake with sequence numbers

Note

The sequence numbers used by TCP allow a destination device to easily reassemble incoming messages if they are received out-of-order compared to the order they were sent onto the network.

- If the sender does not receive an ACK packet from the intended destination host, after a while, the sender will attempt to retransmit the same message, repeating the process to ensure the message is delivered successfully.
- So, it provides guaranteed delivery of messages.
- When both hosts are no longer transmitting data between themselves over the network, TCP will attempt to terminate the connection using a four-step process, as shown here:

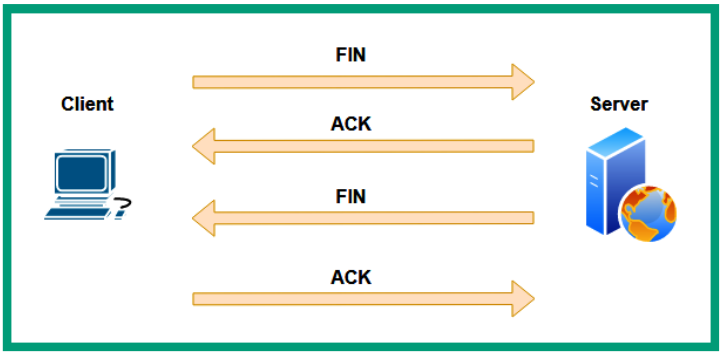


Fig. – TCP terminating a connection

Advantages of TCP

- Establishes a session such as the TCP three-way handshake before exchanging data.
- Provides reliability in delivering data over a network.
- Delivers data using the same order of delivery from the sender to the receiver.
- Uses flow control by creating a window size that has been mutually agreed upon between the source and destination hosts.

Disadvantages of TCP

- There is more overhead on a network as there is an ACK message for every message sent.
- TCP wait for acknowledgment messages from the receiver before sending more data. This creates a delay in the delivery of the messages.

User Datagram Protocol

- It is a connectionless protocol that does not establish a logical connection between the source and destination devices.
- It does not provide any guarantee of delivery of messages over a network.
- It does not provide any acknowledgments when messages are delivered, so the sender does not know whether the messages were delivered or not.
- This makes UDP an unreliable Transport layer protocol within the networking model.
- As the sender device does not use sequence numbers, there is no way to determine how to properly reassemble the messages in their correct order.

Advantages

- Since it does not wait for any acknowledgment from the destination host, it is faster. It is beneficial for application layer protocols that are time-sensitive such as Voice over IP (VoIP) and Video over IP solutions that are used in real time.
- Low overhead on the network since no acknowledgment messages are returning to the sender.

Disadvantages

- It does not provide reliability in delivering data over a network.
- It may not deliver the message to the receiver in the same order as send by the sender.
- It does not use flow control.

Network layer

- The Network layer of the OSI model is responsible for ensuring the logical addressing information is inserted into the datagram.

- Each device requires a unique Internet Protocol version4 (IPv4) or Internet Protocol version 6 (IPv6) address that allows them to communicate with devices on their local and remote networks.
- The Network layer encapsulates a layer 3 header onto the datagram by inserting the source and destination IP addresses.
- Once the PDU from the Transport layer is encapsulated with the layer 3 header, it is referred to as a **Packet**.

The following diagram shows a high-level overview of a client sending a message to a server:

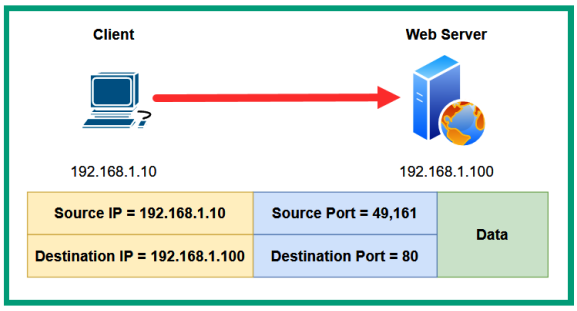


Fig. – Packet header

- Additionally, the Network layer is responsible for the routing services that occur on the network.
- Devices such as routers are considered to be layer 3 devices that can interconnect different networks and forward packets between networking using the information within the layer 3 headers of the packet, such as the destination IP address.
- Each time a router on the network receives a packet, it checks the destination IP address within the layer 3 headers of the packet and the routing table on the router to determine whether a valid route to the destination exists.
- Therefore, a sender must insert the accurate layer 3 addressing (IP addresses) onto the layer 3 header of the packet to ensure networking devices such as routers can forward the packet to the destination.

Note

The source IPv4 address on a packet may change due to the **Network Address Translation** (**NAT**) operating on a router.

Internet Protocol (IP)

- It is a connectionless layer 3 protocols that does not establish any logical connection or session between the sender and receiver of the message.
- If packets are lost or corrupted during the transmission process, the messages are not retransmitted.
- It is unreliable and does not provide any guarantee that the data will be delivered to the destination host.
- However, it provides low overhead on the network as a connectionless protocol.
- The IP indicates to the Transport layer whether or not to use the TCP, UDP, or other protocols in its header information.

Data Link layer

- It is responsible for moving the datagrams from the upper layers onto the actual network.
- It handles flow control.
- It encapsulates a layer 2 header and trailer onto the packet, creating a **frame**.
- It handles error detection to identify whether any incoming frames from the physical network are corrupted and discard them.

Within the Data Link layer, two sublayers assist with ensuring frames are encapsulated, de-encapsulated.

- **Logical Link Control (LLC)**
- **Media Access Control (MAC)**

Logical Link Control

- It is responsible for ensuring there is communication between the networking applications, software, and protocols of the upper layers of the OSI model and the local host's device hardware such as the Network Interface Card (NIC).
- It inserts information within theframe, which indicates the network layer protocol that is being used within the frame.
- Additionally, it allows many layer 3 protocols such as IPv4 and IPv6 to use the same network media and device.

Media Access Control

- It is responsible for performing the data encapsulation process and controlling access to the network device such as the NIC and networkmedia (wired, wireless, or fiber optic).
- It is also responsible for inserting the layer 2 physical addressing information (MAC Address) onto the layer 2 header of the frame.
- It handles error detection by inserting a trailer **Frame Check Sequence (FCS)** into the frame.

  The following diagram shows a high-level overview of the layer 2 header and trailer of a frame:

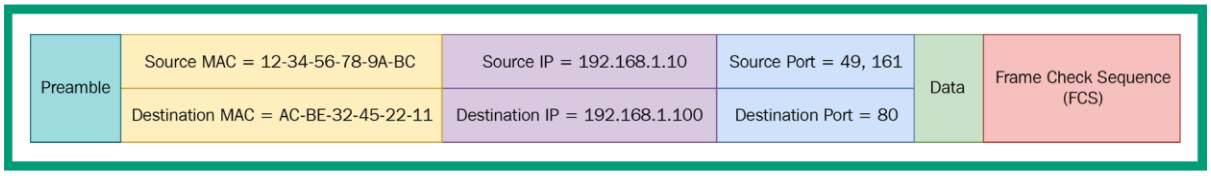


Fig. – Frame

As shown in the preceding diagram, the frame is encapsulated with a layer 2 header that contains the source and destination MAC addresses, as well as the preamble, which is used to identify the start of the frame with sequencing and synchronization.

- The **preamble** helps the receiver of the message determine where the frame begins and how to reassemble the message in the correct order.

- The minimum support frame size on a network is 64 bytes, while the maximum support size is 1,518 bytes of all the contents, including the addressing headers, trailer, and data, excluding the preamble.

A MAC address is a **48-bit** address that is embedded onto a NIC by the vendor of the device. The 48-bit (6-byte) binary MAC address is usually written in hexadecimal (ranges 0 – 9, A – F) to easily identify an address apart from another.

The first 24 bits (3 bytes) of a MAC address is known asthe **Organizationally Unique Identifier (OUI)** as it is assigned by the vendor of the device/NIC.

- The OUI portion of a MAC address can determine the type/vendor of a device that is connected to a network.
- The last 24 bits (3 bytes) of the MAC address are uniquely addressed.

The following diagram shows an example of the OUI portion of a MAC address:

Organizationally Unique Identifier (OUI)	Assigned by the Vendor
3 Bytes	3 Bytes
24 Bits	24 Bits
00-60-5C	3d-d9-01
Cisco Systems	Device-Specific

Fig. – The OUI portion of a MAC address

MAC addresses are usually presented a bit differently based on the vendor of the device or operatingsystem. The following are examples of the same MAC address in different formats:

- 0060.5c3d.d901: This format is usually used by Cisco systems
- 00-60-5c-3d-d9-01: This format is commonly used on Microsoft Windows operating systems
- 00:60:5c:3d:d9:01: This format is found on Linux-based systems.

Physical layer

- When the Data Link layer places the small blocks of data (bits) onto the physical network media, they are converted into electrical signals that are sent through media such as fiber optic, cable, or the air.
- It defines various standards and frameworks that describe how data can travel over the network media types.
- It is simply the electrical wires, mediatype, and even the connections such as ports and interfaces on a network.
- Each physical component on a network needs a set of rules on how to send and receive data over the physical network.

It addresses the following elements to ensure data can be sent over a network:

- Physical components
- Encoding
- Signaling

The **physical components** are the hardware elements that you see on a network, such as the networking devices, the physical interfaces/ports on a device, the networking cables that are used to interconnect devices, and so on. These uses a set of standards to ensure devices can transmit messages over the network.

The **encoding process** describes the processes or methods used by a device to convert a stream of messages, such as bits, into code. This code is used to represent patterns that are recognizable by both the sender and receiver devices over the network.

The **signalling** element of the Physical layer describes how the signals are created and placed on the physical network media by a sender device. The signals that are generated by the sender are electrical,wireless, or even optical (light), depending on the network media that is connected to a device.

Understanding TCP/IP

- It was developed by US DoD
- The TCP/IP protocol suite is a group of networking protocols that all work together to ensure messages can be exchanged over any type of network between sender and receiver devices.
- Its original version consists of 4 layers.
- Modern versions of the TCP/IP protocol suite have five layers, splitting the bottom layer into Physical and Data Link layers.

The following diagram shows a comparison of the original TCP/IP protocol suite and the OSI model:

<table>
<tr><th>Layer</th><th>OSI Model</th><th>TCP/IP</th><th>Layer</th></tr>
<tr><td>7</td><td>Application</td><td rowspan="3">Application</td><td rowspan="3">4</td></tr>
<tr><td>6</td><td>Presentation</td></tr>
<tr><td>5</td><td>Session</td></tr>
<tr><td>4</td><td>Transport</td><td>Transport</td><td>3</td></tr>
<tr><td>3</td><td>Network</td><td>Internet</td><td>2</td></tr>
<tr><td>2</td><td>Data Link</td><td rowspan="2">Network Access</td><td rowspan="2">1</td></tr>
<tr><td>1</td><td>Physical</td></tr>
</table>

Figure – TCP/IP protocol suite

As shown in the preceding diagram, the following are the four layers of the TCP/IP protocol suite:

- **Application**
- **Transport**
- **Internet**
- **Network Access**

- The Application layer of TCP/IP absorbs all the functionality and responsibilities of the Application, Presentation and Sessions layers of the OSI model.
- The Transport layer of both the OSI model andTCP/IP has the same functionalities and responsibilities.
- The Internet layer of TCP/IP is equivalent to the Network layer of the OSI model.
- The Network Access layer of TCP/IP is equivalent to both the Data Link and Physical layers of the OSI model.

Note

The **Network Access** layer of TCP/IP is sometimes referred to as the **Link** layer or the **Network Interface** layer.

The following diagram provides a high-level overview of a computer sending a message that contains data to a server using the TCP/IP protocol suite:

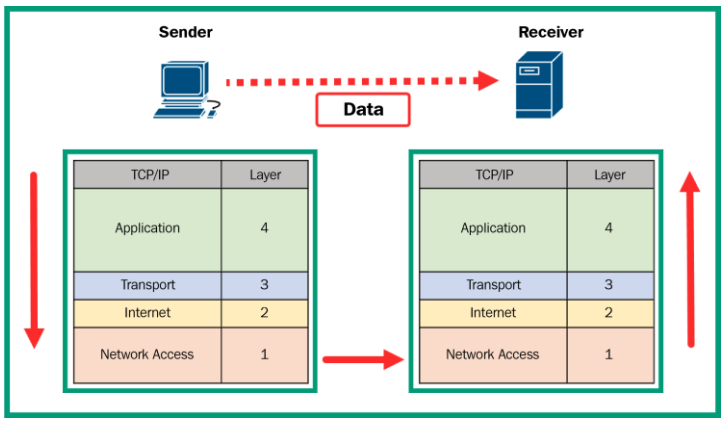


Fig. – TCP/IP protocol suite

Important note

As data moves down a networking model, such as the OSI model or TCP protocol suite, each layer encapsulates a header containing addressing information. When a device receives a message over a network, the process is reversed as each layer de-encapsulates the headers and the message moves up to the Application layer.

Data encapsulation concepts

It's important to understand the variousfields found within Ethernet, IPv4, IPv6, TCP, and UDP headers.

Ethernet header

At the Data Link layer, when a packet is received from the Network layer, it is encapsulated with alayer 2 header and trailer. The following diagram shows each field within an Ethernet header:

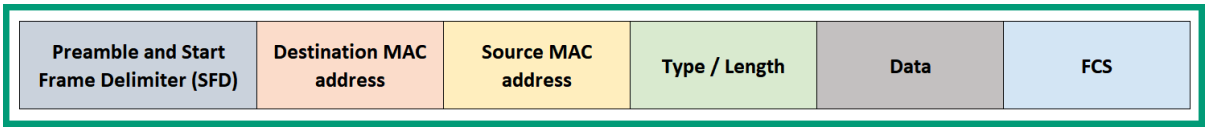


Fig. – Ethernet header

The following are the roles and functions of each field found within an Ethernet header:

- **Preamble** and **SFD**: The preamble is made up of 7 bytes and the **Start Frame Delimiter** (**SFD**)is 1 byte in size, so the entire field is a total of 8 bytes in size. This field is used to synchronize messages being transmitted between a sender and receiver over a network. This field is also used to indicate the start of the frame to the receiver.
- **Destination MAC address**: This field is 48 bits (6 bytes) in length and contains the layer 2 physical address (MAC address) of the next device to receive the message.
- **Source MAC address**: This field is 48 bits (6 bytes) in length and contains the layer 2 physical address of the sender of a frame.
- **Type / Length**: This field is 2 bytes in length and contains details that are used to identify the upper layer protocol (IPv4, IPv6) that is encapsulated within a frame.
- **Data**: The data field ranges between 46 to 1,500 bytes and contains the data from the Network layer of the networking model. All Ethernet frames are required to be at least 64 bytes in length. If the frame is less than 64 bytes, additional bits, known as a *pad*, are inserted to increase the size of the frame to the minimum length.
- **FCS**: The **Frame Check Sequence** (**FCS**) field is made up of 4 bytes in length and it's used to verify the integrity of a frame and detect errors.

IP headers

The following diagram shows the field within an IPv4 header:

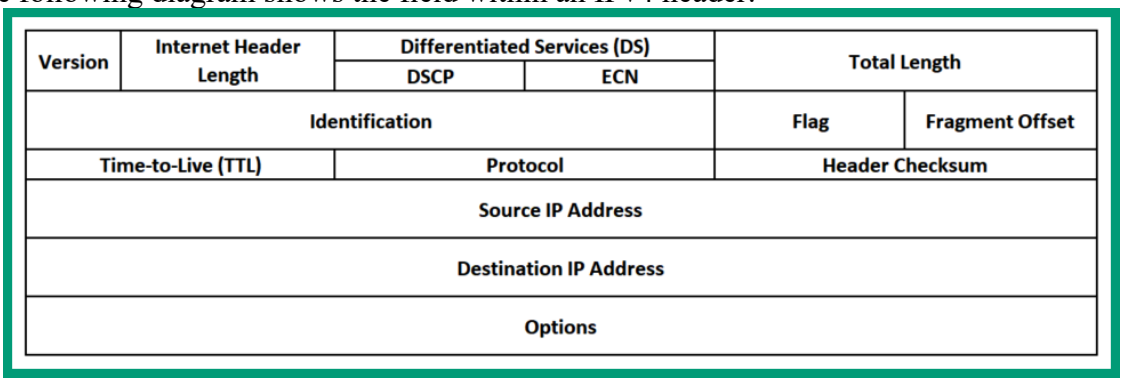


Fig. – IPv4 header

The following is a description of each field within an IPv4 header:

- **Version**: This field is made up of 4 bits and is used to identify the message as an IPv4 packet.
- **Internet Header Length**: This field is made up of 4 bits and is used to indicate where the header section ends and the data section starts.
- **Differentiated Services** or **DiffServ**(**DS**): This field is made up of 1 byte (8 bits) and is used to determine the priority of the packet on the network. Within the DS field, the 6 most significant bits (from the left to right in a binary number) are used to present the **Differentiated ServiceCode Point** (**DSCP**), while the 2 least significant bits (from right to left in a binary number) are used to represent the **Explicit Congestion Notification** (**ECN**) details.
- **Total length**: This field is made up of 16 bits (2 bytes) and is used to indicate the total size of the IPv4 packet.
- **Identification**: This field is made up of 16 bits (2 bytes) and is used to provide identification numbering to each fragmented packet that belongs to an original message.
- **Flags**: This field is made up of 3 bits and is used to indicate whether the packet is to be fragmented or not.
- **Fragment offset**: This field is made up of 13 bits and is used to indicate the sequencing position of a fragmented packet.
- **Time To Live** (**TTL**): The TTL field is made up of 1 byte (8 bits) and is used to determine the life of the packet as it is transmitted between a sender and receiver over the network.
- **Protocol**: This field is made up of 1 byte (8 bits) and is used to indicate the payload type (ICMP/TCP/UDP) that is enclosed within the packet.
- **Header checksum**: This field is made up of 2 bytes (16 bits) and is used to determine whether there's any corruption within the IPv4 header.
- **Source IP address**: This field contains the source IPv4 address of the sender, which is 32 bits(4 bytes) in length.
- **Destination IP address**: This field contains the destination IPv4 address of the intended recipient, which is 32 bits (4 bytes) in length.
- **Options**: This field is optional as it's not always used.

The Network and Internet layers can also be encapsulated within an IPv6 header on the segment tocreate a packet. The following are the fields within an IPv6 header:

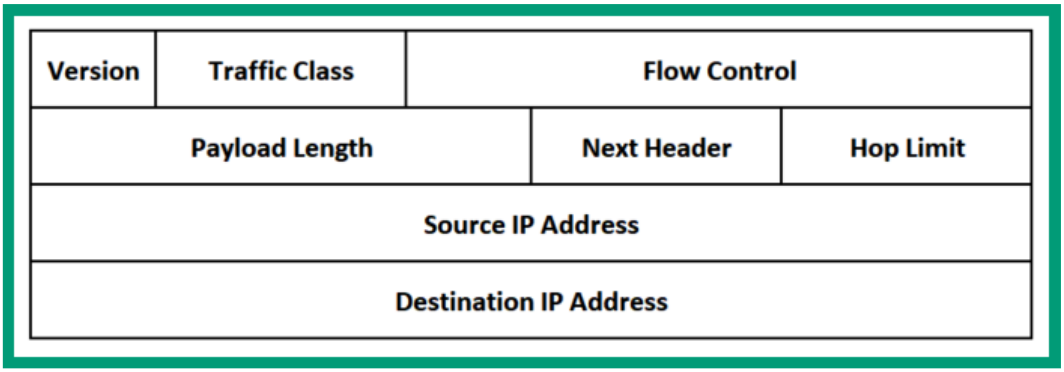


Fig. – IPv6 header

The following is a description of each field found within an IPv6 header:

- **Version**: This field is 4 bits in length and is used to identify this packet as an IPv6 packet on the network.
- **Traffic class**: This field is 8 bits (1 byte) in length. It has the same functionality as the DS field found within an IPv4 packet.
- **Flow control**: This field is 20 bits in length and is sometimes referred to as the **Flow Label**.This field is used to inform the routers on the network to use the same type of handling for IPv6 packets that has the same flow control/flow label information.
- **Payload length**: This field is 16 bits (2 bytes) in length. It is used to represent the length of the enclosed data or payload in the IPv6 packet.
- **Next header**: This field is 8 bits (1 byte) in length. It is used to indicate the payload type that is (TCP/UDP) enclosed within the IPv6 packet.
- **Hop limit**: This field is 8 bits (1 byte) in length and it has the same role and functions as the TTL field found within an IPv4 packet.
- **Source IP address**: This field contains the 128-bit IPv6 address of the sender.
- **Destination IP address**: This field contains the 128-bit IPv6 address of the receiver.

TCP header

The following diagram shows the fields within a TCP header:

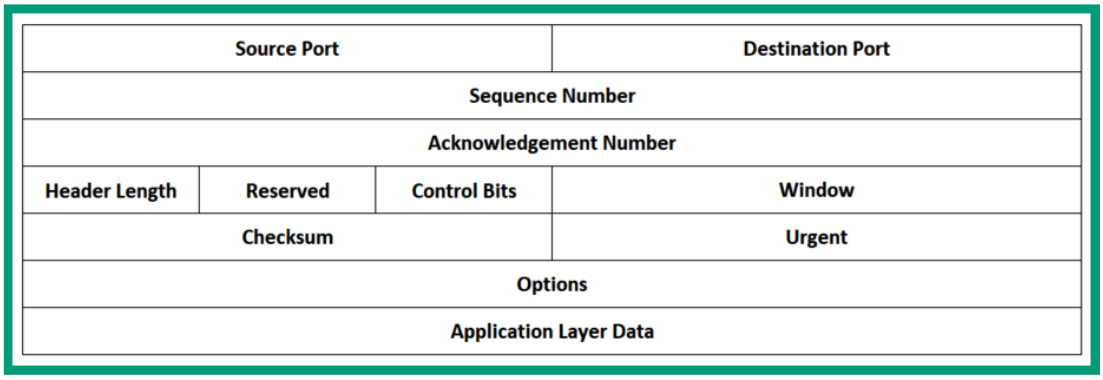


Fig. – TCP header

The following is a description of each field within a TCP header:

- **Source port**: This is a 16-bit (2-byte) field that contains the source service port number of the source application layer protocol.
- **Destination port**: This is a 16-bit (2-byte) field that contains the destination service port number for the destination application layer protocol.
- **Sequence number**: This is a 32-bit (4-byte) field that is used during the reassembly process on the receiver device.

- **Acknowledgment number**: This is a 32-bit (4-byte) field that is used to indicate that the message(data) has been received. This value will be the sequence number + 1.
- **Header length**: This is a 4-bit field that is sometimes referred to as the **data offset** field. It indicates the length of the TCP header.
- **Reserved**: This is a 6-bit field reserved for future usage.
- **Control bits**: This is a 6-bit field that is used to specify various TCP flags such as URG, ACK, PSH, RST, SYN and FIN. These are sometimes referred to as the **Flag** field.
- **Window**: This is a 16-bit (2-byte) field that indicates the number of bits or bytes that can be accepted during data transmission between a sender and receiver.
- **Checksum**: This is a 16-bit (2-byte) field that is used to detect any errors within the TCP header.
- **Urgent**: This is a 16-bit (2-byte) field that is used to indicate urgency on the TCP header.
- **Options**: This is an optional field within the TCP header that can range between 0 and 320 bits in length.
- **Application layer data**: This field contains the data that's been received from the application layer protocol.

The following six TCP flags are found within the control bit field within a TCP header:

- **URG**: Indicates urgency on the TCP segment
- **ACK**: Indicates acknowledgment of a message
- **PSH**: Performs the push function
- **RST**: Used to reset a connection
- **SYN**: Indicates a synchronization message with a synchronization sequence number
- **FIN**: Indicates to gracefully terminate (finish) a session

UDP headers

Many use the User Datagram Protocol (UDP) to ensure low overhead and faster transmission. The following diagram shows the fields within a UDP header:

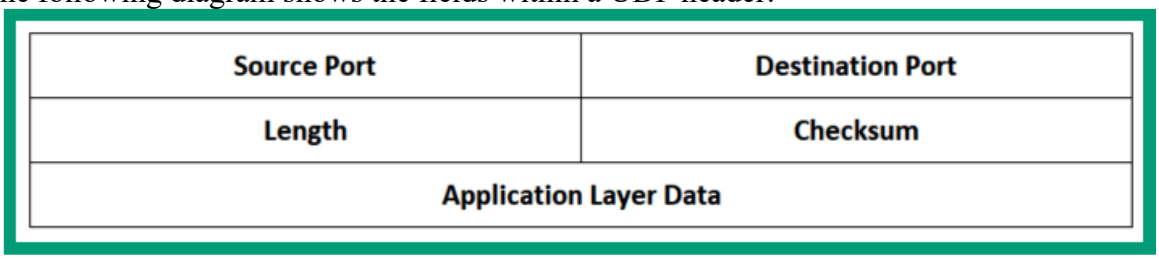


Fig. – UDP header

The following is a description of each field within a UDP header:

- **Source port**: This is a 16-bit (2-byte) field that contains the source service port number of the source application layer protocol
- **Destination port**: This is a 16-bit (2-byte) field that contains the destination service port number for the destination application layer protocol
- **Length**: This is a 16-bit (2-byte) field that indicates the total length of the UDP datagram.
- **Checksum**: This is a 16-bit (2-byte) field that is used for detecting any errors within the UDP header.
- **Application layer data**: This field contains the data that's been received from the application layer protocol.